Weishu Zhang

626-566-4310 | weszhang@berkeley.edu | LinkedIn | GitHub | Website

Available December 2026. Backend-leaning full-stack engineer building production agent systems and enterprise workflow runtimes.

Education

University of California, Berkeley Berkeley, CA
Bachelor of Science in Electrical Engineering and Computer Sciences *May 2027*
- GPA: 4.0/4.0
- Relevant Coursework: Efficient Algorithms, Network Science, Artificial Intelligence

Technical Skills

Languages: Python, TypeScript/JavaScript, SQL, Java, Go, C/C++
Full Stack: React, Next.js, Node.js, FastAPI, PostgreSQL, Redis, SQLite, REST APIs
AI & Agents: LangGraph, MCP, LLM APIs, agent runtimes, structured generation, PyTorch, vLLM, QLoRA
Infrastructure: Docker, Kubernetes, gVisor, OCI, OpenTelemetry, pytest, distributed jobs, CI/CD

Experience

Amazon
Incoming Software Development Engineer Intern *Sep 2026 – Nov 2026*

ASML San Jose, CA
Software Engineering in Test Intern | Blog *2026 – Present*
- Built RRI for daily SQA regression tracking, cutting 80% of repetitive review work by automating Confluence review pages, Jira metadata, qTest sync, email, and compute-job follow-up.
- Designed a Test Plan Runtime Agent that drafts plans from cases covering 20+ coverage types, so QA only authors a few exceptional cases instead of handwriting the full plan.
- Compressed test-plan turnaround from 2 weeks of manual writing to as little as hours of human review, with the agent executing approved jobs asynchronously and writing results back.

Ludus Remote
Co-Founder & Software Engineer, Cloud Agent Infrastructure *2026 – Present*
- Built a Kubernetes-native sandbox control plane that packs 15+ concurrent gVisor-isolated agent instances onto a single AWS m6i.2xlarge (8 vCPU, 32 GB) via bin-packing, warm pools, and suspend/resume.
- Hardened untrusted-code execution with gVisor/Kata RuntimeClasses, namespace RBAC, default-deny NetworkPolicies, and CPU, memory, and ephemeral-storage quotas.
- Standardized reproducible agent environments with immutable OCI images, PVC-backed workspaces, TTL garbage collection, and OpenTelemetry traces so idle sandboxes yield host CPU.

Berkeley Artificial Intelligence Research (BAIR) Berkeley, CA
Undergraduate Research Assistant *Jan 2026 – Present*
- Led expert task acquisition for the Agent Human Last Exam benchmark, recruiting 30+ domain experts and contributing to the collection of 80+ complex, cross-domain agent tasks.
- Built an LLM-powered proposal agent that converts rough expert ideas into structured benchmark tasks, supporting iterative refinement, constraint checking, and higher-quality submissions.

University of California, San Francisco San Francisco, CA
Software Research Assistant *Aug 2025 – Present*
- Built a production-oriented long-context clinical information extraction pipeline, increasing inference throughput by 3x and reducing VRAM usage by approximately 80% using vLLM, paged attention, quantization, and batched execution.
- Created a 4,000+-example domain dataset, fine-tuned Qwen models with QLoRA, and implemented validation and self-consistency checks that maintained 100% schema-valid structured outputs.

Open Source

PyTorch | *torch.compile, torch.export, Dynamo* PRs
- Contributed production fixes to pytorch/pytorch across compiler and export internals, resolving eager-versus-compiled correctness gaps, tracing failures, and export regressions with targeted tests and maintainer-reviewed patches.

Selected Projects

mini-vLLM | *Python, PagedAttention, KV cache, Scheduler* Code
- Built a GPU-free from-scratch model of vLLM's systems core: paged KV cache, continuous batching, preemption, and prefix caching, with reproducible benchmarks (7x more active sequences, 1.53x throughput, 54x lower p99 TTFT).

GradeView | *React, TypeScript, Python, PostgreSQL, Redis* Live | Demo | Poster
- Refactored a production Berkeley course-analytics platform into containerized React, Node.js, and Python services with PostgreSQL as source of truth. Pages that previously took 10+ seconds now load in tens of milliseconds after Redis cache prewarming.